From enterprise data to production AI

Why most AI work stalls before production, and the pipeline that ships it

M Usman

An applied view of the data-to-AI pipeline.

Where this is heading

The demand is real. Delivery is what breaks.

M Usman · From enterprise data to production AI.

The gap is not ambition, it is delivery

- AI and big-data roles are the **fastest-growing to 2030** (WEF, 2025), and the measured global AI wage premium has reached **56%** (PwC, 2025). The intent and the budget are there.
- The ambition stalls before it reaches a decision:
  - Only about **48%** of AI projects reach production, and a prototype takes about **8 months** to get there (Gartner, 2024)
  - At least **30%** of generative-AI projects are expected to be **abandoned after proof of concept** by the end of 2025 (Gartner, 2024)
  - **More than 80%** of AI projects fail, around twice the rate of non-AI IT projects (RAND, 2024)

> The bottleneck is rarely the model. It is the pipeline around it.

The slow step

AI fails in the plumbing, not the algorithm

The value is lost in the distance between raw data and a decision: ungoverned data, brittle pipelines, evaluations that flatter the model, and a deployment nobody is watching. Security is the **most-cited technical challenge** to AI adoption (Anaconda, 2024: 42%).

> The slow step is the distance between a notebook that works once and a governed, monitored system that ships repeatably. The four stages that follow are where that distance is closed.

The pipeline, end to end

Input: *raw enterprise data*

1 · Foundation governed, AI-ready data · contracts · the semantic layer · lineage	**guard:** the data swamp
2 · Pipeline ingest · transform · data-quality tests as code · orchestration	**guard:** silent breakage
3 · Model features · leakage-safe evaluation · the metric the decision needs	**guard:** leakage and vanity metrics
4 · Serving versioned API · drift and performance monitoring · retraining	**guard:** the model nobody is watching

Output: *a monitored decision you can trust*

Each layer has one job, one common way to lose the value, and one discipline that protects it. The next four slides take them in turn.

M Usman · From enterprise data to production AI.

1 · The foundation: governed, AI-ready data

A trustworthy, governed, modelled layer, so everything downstream reads data with a known shape, owner, and meaning.

Practical steps

- Resolve core entities into master records with one authoritative key
- Write a **data contract** per dataset: schema, owner, freshness SLA, change policy
- Define the **semantic layer** once: entities, relationships, metric formulae
- Attach governance and lineage to the data at source, not to the report

The pitfall: the data swamp. Ungoverned data; the model silently inherits duplicate entities, ambiguous fields, and three versions of "revenue". Invisible here, costly later.

What good looks like. A dataset is not "available" until it has a contract, an owner, and a place in the semantic layer.

Illustrative tools: dbt semantic layer, Cube · DataHub, OpenMetadata, Collibra · schema registries.

2 · The pipeline: ingest, transform, test

Source data moved into layered, tested, business-ready tables under an orchestrator that manages dependencies, retries, and alerts.

Practical steps

- Land raw (bronze), clean and conform (silver), model for consumption (gold)
- Enforce the Stage-1 contract on the way in: deduplicate, type-cast, standardise
- Write **data-quality tests as code** beside the transforms, tiered to each layer
- Orchestrate as a dependency graph with freshness, volume, and schema monitors

The pitfall: silent breakage. An upstream column is renamed or a load arrives half-empty; with no contract test the pipeline fails at 3am or, worse, propagates corrupted data.

What good looks like. Quality is versioned with the transforms and runs every time; lineage points straight to the broken node. Minutes to root-cause, not hours.

Illustrative tools: dbt, SQLMesh · Great Expectations, Soda · Dagster, Airflow, Prefect · Monte Carlo.

3 · The model: features and honest evaluation

Governed data turned into a model, where production value is decided not by the algorithm but by whether the reported score is the score you will see in production.

Practical steps

- Split train, validation, test **first**; fit all preprocessing on the train fold only
- Match the protocol to the data: stratified, grouped, or time-ordered splits
- Choose the metric from the decision: precision and recall, F1, or PR-AUC, not raw accuracy
- Resample **inside** each fold; lock an untouched test set; audit features for target leakage

The pitfall: leakage and vanity metrics. Information from the future or the target slips into training; the score is spectacular and fictional, then collapses in production.

What good looks like. The whole feature-and-resample chain sits inside the cross-validation loop, fitted per fold. Treat a result that looks too good as an alarm.

Illustrative tools: scikit-learn (Pipeline, StratifiedKFold, GroupKFold, TimeSeriesSplit) · imbalanced-learn · MLflow · SHAP.

4 · Serving and monitoring: ship it, then keep it honest

The model packaged behind a versioned interface, deployed into the decision path, and watched, so silent decay is caught before it does damage.

Practical steps

- Package model, features, and dependencies together; verify train-serve parity
- Release progressively: shadow, canary, or A-B, with a fast rollback path
- Monitor three things: input **data drift**, **concept drift**, and **ground-truth performance**
- Trigger retraining on a drift threshold or a measured drop, not the calendar alone

The pitfall: the model nobody is watching. It stalls in a notebook, or ships without monitoring and decays silently as the world drifts, found out only when a business metric moves.

What good looks like. Monitoring is part of the build. A model is not "done" until drift detection, performance tracking, and a retraining trigger are wired in.

Illustrative tools: FastAPI, BentoML, KServe · Docker, Kubernetes · Evidently, Arize, NannyML · a model registry.

The readiness ladder

5 · Governed, monitored, self-correcting
end-to-end lineage; data, drift, and performance monitoring; automated retraining; sign-off gate

4 · Shipped and served
leakage-safe models behind versioned APIs; canary release with rollback; train-serve parity

3 · Modelled and tested
contracts and semantic layer; medallion pipeline with quality-as-code; orchestrated, in CI

2 · Scripts and schedules
logic on a cron; no tests, no contracts, no lineage; users find the breakage

1 · Notebooks and heroics
one laptop, manual files; nothing versioned or reproducible

The four stages above are the climb from rung 2 to rung 5: value compounds upward, and most teams sit at 2 to 3 while believing they are at 4.

Four pitfalls that span the whole pipeline

- **No end-to-end lineage.** A broken metric becomes manual archaeology across the graph, not a field traced to its root in minutes.
- **Governance bolted on last.** Retrofitted access, classification, and quality are brittle and slow; attached at source, they are enforced for free downstream.
- **Evaluating on leaked data.** The most expensive systemic error: every metric turns fictional, and the only tell is a score too good to be true.
- **Shipping without monitoring.** A depreciating asset nobody is watching, discovered through a business metric long after the damage is done.

The lens behind this view

My view comes from two vantage points usually held apart. One is years building and governing enterprise data estates across multiple ERP stacks, where master data, the modelled layer, and BI are real problems rather than diagrams. The other is doctoral work on efficient, proactive AI, where honest evaluation and the cost of a wrong prediction are the whole job.

The four stages above are where I have watched value get made, and lost.

> Four worked examples, with code-reproduced numbers, are public: github.com/mtusman-ai

In one line

AI does not fail in the model; it fails in the pipeline around it. Govern the foundation, test the flow, evaluate honestly, and ship behind a monitored interface. The work that stalled in a notebook becomes a system that runs.

Sources

WEF Future of Jobs (2025) · PwC Global AI Jobs Barometer (2025): 56% global AI wage premium · Gartner (May 2024): 48% reach production, ~8 months prototype-to-production · Gartner (2024): ≥30% of GenAI projects abandoned after PoC by end-2025 · RAND, The Root Causes of Failure for AI Projects (2024): 80%+ fail · Anaconda State of Data Science (2024): 42% cite security · Engineering practice draws on the data-contracts, medallion, ML-evaluation, and MLOps literature: dbt; Google Rules of ML and ML Test Score; Monte Carlo data-observability.

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman